The Harness Is the Attack Surface: A Security Analysis of AI Coding-Agent Harnesses (2025–2026)

Thanatchon Chongmankhong
Independent researcher
https://github.com/thanatchon36/harnesswatch

***Abstract*—Coding agents are built as *model plus harness*: the runtime loop that holds credentials, runs shell commands, mounts Model Context Protocol (MCP) servers, reads repository configuration, spawns sub-agents and runs headless in CI. We study the security of this deterministic layer across 58 harnesses and runtimes in four method tiers: open source (white-box), closed but runnable (black-box), closed and unreachable (documentation only), and SDK runtimes. We contribute: (i) a threat taxonomy (T01–T19) mapped to STRIDE, MITRE ATLAS and the OWASP LLM, Agentic and MCP lists (RQ1); (ii) a 15-dimension control matrix with 664 evidence-linked scores and a second LLM-agent rating on the same fact base (quadratic $\kappa = 0.88$) (RQ2); (iii) a survey of 323 publicly disclosed harness-level advisories and incidents from 2025–2026: in 48% the root cause is a flawed approval, allowlist or path check or configuration honoured before consent, while pure prompt injection, the residual class of our procedure, is 5% (RQ3); (iv) `harnesswatch`, an open-source auditor with micro precision 0.983 (95% CI 0.950–0.994) on a held-out synthetic corpus and a stratified prevalence study of 1010 public repositories (RQ4); (v) an inert canary benchmark (350 runs, one small local model) that found no detectable difference in injection-following between permission modes but showed a documented Claude Code import feature placing a gitignored secret in the session transcript in 10 of 10 runs (RQ5); (vi) a governance checklist mapped to NIST AI RMF, ISO/IEC 42001, OWASP and the MAS AI risk-management guidelines (RQ6); and (vii) a black-box comparison of vendor claims with observed behaviour, where 24% of 117 documented claims were testable headless and 22 of the 28 tested matched (RQ7). Every result number is generated from versioned results files.**

***Index Terms*—AI coding agents, agent harness, prompt injection, Model Context Protocol, software supply chain, security controls**

I. Introduction

Answers the motivation for RQ1–RQ7 and summarises the contributions.

In 2025 and 2026 coding agents moved from chat windows into terminals, IDEs and CI pipelines. Vendors and open-source projects converged on a common architecture: an agent is a *model* plus a *harness*. The harness is the program around the model. It assembles context from the repository, exposes tools (shell, file edit, web fetch, MCP servers), decides which tool calls need human approval, stores credentials and transcripts, spawns sub-agents, and can run unattended in CI. The model proposes actions; the harness carries them out.

Most security research on LLM agents studies the model side, i.e. whether a model follows injected instructions [1]–[3], and treats the tool-execution layer as fixed. Yet the harness decides whether an injected instruction becomes a shell command, whether a cloned repository can run a hook before the user consents, whether an MCP server starts from a cloned file, and whether a secret lands in a world-readable log. These are deterministic software decisions that can be specified, tested and audited like any access-control code.

Public advisories point the same way. Of the 323 verified harness-level advisories and incidents we collected for 2025–2026 (Section VII), 104 stem from flawed approval, allowlist or path checks and 52 from workspace configuration honoured before consent; only 17 are pure prompt injection. This population is filtered towards software defects (model failures rarely receive CVEs) and RC1 is the residual class of our procedure, so these shares describe disclosed harness bugs, not overall risk; within that scope, injection is often the vector while the damage depends on harness controls.

Comparing harnesses is hard: the field changes monthly, many popular harnesses are closed source, and documentation describes intended rather than actual behaviour. We therefore organise the study by *method*: white-box for open source, black-box observation for closed harnesses we can run, documentation for those we cannot, and a separate track for SDK runtimes (Section III).

Research questions: **RQ1**: What is the attack surface of a harness and how does it differ from classic LLM-application risk? **RQ2**: How do harnesses differ in their security controls? **RQ3**: Which harness-level incidents were disclosed in 2025–2026, and with which root causes? **RQ4**: Can dangerous configuration be detected automatically across harness formats? **RQ5**: How resilient are harness $\times$ model $\times$ permission-mode combinations to indirect prompt injection and secret residue, measured with inert canaries? **RQ6**: Which controls should an organisation mandate before adopting harnesses? **RQ7**: For closed harnesses, do documented controls match observable client behaviour?

Contributions: A harness threat taxonomy with framework mappings (RQ1, Section IV); an evidence-linked control matrix of 58 harnesses and runtimes (RQ2, Section V); a black-box claims-versus-observed method for closed harnesses (RQ7, Section VI); a two-source incident survey (RQ3, Sec-

tion VII); `harnesswatch`, an Apache-2.0 auditor, with accuracy, prevalence, canary-bench and residue experiments (RQ4, RQ5, Sections VIII–IX); and a governance checklist (RQ6, Section X). `make all` rebuilds every table, figure and result number from versioned results (Appendix A).

II. Background: Anatomy of a Harness

Answers the descriptive part of RQ1 and sets up RQ2.

The loop: A harness builds a context, calls the model, checks each proposed tool call against a permission policy, executes it and feeds the result back; repository content, tool output and MCP results flow in, commands and requests flow out.

Repository files: Harnesses auto-load instruction files (`AGENTS.md`, `CLAUDE.md`, `.github/copilot-instructions.md`, `.cursor/rules`), settings that can set environment variables, permission modes and *hooks* (shell commands bound to lifecycle events), and MCP declarations (`.mcp.json`, `.vscode/mcp.json`). Formats are shared: Grok Build imports Claude Code and Cursor hook and MCP files[D] [4], [5]. A cloned repository is therefore both data and configuration.

Permissions and sandbox: Modes range from "ask before every tool call" to "approve everything" (YOLO, bypass, full-auto, always-approve), with allow and deny rules over commands and paths. Several harnesses now default to a model-based approver; Claude Code starts `claude -p` in a classifier-mediated `auto` mode when feature flags are not fetched[D][T] [6]. Some confine commands with an OS sandbox (Seatbelt, bubblewrap, Landlock) or a container[D] [7]. Approval decides *whether* an action runs; the sandbox limits *what it reaches*.

Extensions, state and headless mode: MCP [8] servers are launched locally (often via unpinned `npx` or `uvx`) or reached remotely, and their tool descriptions enter the context verbatim; skills and plugins add code and prompts from marketplaces. Credentials, transcripts and persistent memory live in a per-user state directory, and sub-agents may inherit the parent's permissions. The harnesses we benchmarked all run non-interactively[D][T] (`claude -p`, `codex exec`, `grok -p`, `copilot -p`); in CI the triggering issue or pull request is attacker-controllable and the job may hold secrets.

The 2026 landscape: The field splits into open-source harnesses, closed but installable clients, hosted agents and SDK runtimes (Section III). On 31 March 2026 the npm release 2.1.88 of Claude Code was reported to ship a source map exposing client internals[3P] [9]; we cite this only as an event and did not use the exposed material.

III. Scope, Tiers and Evidence Method

Answers how the evidence behind RQ2–RQ7 was collected.

Tiers by method: We split harnesses by what can be observed, not by market share. *Tier A* (open source, white-box): we read the source at a pinned commit and run the client in containers (e.g. Codex CLI, OpenCode, Gemini CLI, Qwen Code, Aider, goose, Cline, Hermes Agent). *Tier B* (closed but runnable, black-box): we install the official client in a disposable container, record what it writes and which hosts it contacts, and drive it headless (Claude Code, GitHub Copilot CLI, Antigravity CLI before login). Grok Build is an A/B hybrid: its source is public under Apache-2.0[D] [10], but the public tree lags the npm release (1.0.45 vs 1.0.52) and injects endpoints at build time, so we read the source white-box and treat the shipped npm client as Tier B in E3 and E7. *Tier C* (closed, not reachable): documentation, advisories and third-party research only (e.g. Cursor, Kiro, Devin, Jules). *Tier D* (runtimes and SDKs) is scored on a separate ladder and never ranked against products (the rubric files and figures group it with hosted services as "tier C"). The matrix covers 36 tier A/B products and 22 runtimes and hosted services.

Black-box method: For closed harnesses we install the official release, read its documentation and advisories, parse its configuration, run it under `harnesswatch observe` (Section VIII), probe configuration trust with synthetic repositories carrying hooks, MCP servers and settings, and inspect package metadata. We never decompile, deobfuscate or use leaked source, never decrypt TLS or store request bodies, and stay within vendor terms (otherwise the cell is *N/A (terms)*). No vendor login was placed in any container: closed clients were driven only through documented custom-endpoint features pointing at a local model (Claude Code's `ANTHROPIC_BASE_URL`, although Anthropic states that it "doesn't support routing Claude Code to non-Claude models through any gateway"[D] [11], so our Claude Code runs test harness behaviour in an unsupported configuration, not vendor-supported performance; Copilot CLI's bring-your-own-key mode, which needs no GitHub authentication[D] [12], and Grok Build's custom models[D] [13]). Antigravity has no documented local-model path and its terms restrict use with other products[D] [14], so it was observed only before login.

Evidence labels: Every factual claim about a harness carries one of three labels, shown as superscripts: [T] *verified-by-test*, behaviour or state we observed by running the pinned client; [D] *from-official-docs*, vendor documentation, vendor advisories, registry metadata or the official source code (reading source counts as documentation, not as a test); [3P] *third-party-report*, a write-up by an independent researcher or journalist. Unverified claims are excluded. Documentation and source facts live in the evidence ledger and `matrix/evidence/`; every [T] fact traces to a probe or bench file under `experiments/results/` or `bench/results/`.

Who did the work: Evidence collection, both matrix ratings and incident collection and classification were performed by LLM research agents run and supervised by the author. The two matrix raters scored from the same extracted fact lists, and incidents had a single classifier; Section XII discusses what this means for reliability.

Kinds of finding: We separate a *vulnerability* (a defect that violates the product's own security intent, usually with a CVE or GHSA), a *design weakness* (a documented default or

capability that enables harm without being a bug, e.g. headless mode running repository hooks) and a *policy choice* (e.g. telemetry or retention defaults). Only the first is a bug.

Ethics: Experiments ran in disposable containers on the author's machine with inert canaries and synthetic secrets; the public-repository study stores aggregates only. Potentially unknown issues went to private coordinated disclosure and are not described here. For packages we inspected registry metadata only; one fact obtained by listing strings in a binary was withdrawn as outside this rule.

IV. Threat Model and Taxonomy

Answers RQ1.

Adversaries: (A1) A *content author* controls text the agent reads (README, comment, issue, web page, tool result); (A2) a *repository author* controls cloned files, including configuration; (A3) a *supply-chain attacker* controls a package, MCP server, plugin or release; (A4) a *local or network peer* reaches a service the harness exposes. The defender runs the harness with real credentials, code and host access (the assets). We assume the model sometimes follows injected instructions and that no model-level defence holds.

Taxonomy: We extended the 16-class seed into the classes in Table I. Three classes were added because the incident set contained large groups with no home: sandbox and isolation escape (T17), exposure of the agent's own control plane, such as local HTTP or WebSocket servers without authentication or Origin checks (T18), and conventional software defects in runtimes and SDKs (T19). Each class carries STRIDE letters, MITRE ATLAS techniques, OWASP LLM 2025, OWASP Agentic (ASI01–ASI10) and OWASP MCP Top 10 identifiers; the MCP list is still in beta, and we flag those mappings as unstable. Each class also lists the control dimensions that mitigate it (Section V) and the `harnesswatch` rules that detect it (Appendix C).

Classification procedure: Write-ups mix the entry vector with the control that failed, so we walk the classes in a fixed order and assign the first whose *failed control* the incident exhibits. A parallel 12-question procedure assigns one of twelve root causes, RC1–RC12 (Table III). For example, injection-written configuration that later executes is T02/RC2, not T01; data leaving through a permitted command is T08/RC3. RC1 (untrusted content treated as instructions with no other control failure) is the residual answer reached only when no earlier question applies, so its share is low partly by construction.

How harness risk differs from LLM-application risk: In a chat application, a successful injection yields a bad answer or a leaked system prompt. In a harness, the injected text comes from the developer's own workspace and is acted on with the developer's privileges. Three properties follow. First, *configuration is code*: repository files can define hooks, environment variables and MCP launch commands that run before the model is consulted (T02). Second, *the approval layer is a parser*: allowlists over shell commands must reason about quoting, substitution, redirection and symlinks, and such failures are the largest single group of disclosed incidents (T03, 23% by class; RC3, 32% by root cause). Third, *the harness is a long-lived service*: local servers, sub-agents, memory and CI runners create surfaces that classic LLM threat models do not include (T10, T11, T18). Attack trees for the main paths are in the repository.

V. Comparative Analysis of Security Controls

Answers RQ2.

Rubric: We score 15 control dimensions from 0 to 3: D01 permission model, D02 execution sandbox, D03 network egress, D04 workspace and configuration trust, D05 MCP governance, D06 extensions and plugins, D07 secrets and credential storage, D08 headless and CI safeguards, D09 persistent memory, D10 sub-agent isolation, D11 audit logging, D12 telemetry and retention, D13 distribution supply chain, D14 managed policy, D15 vulnerability disclosure. The scale is 0 absent, 1 opt-in or weak, 2 default-on with documented gaps, 3 strong, documented and centrally enforceable, with per-dimension criteria (Appendix E). Only [D] and [T] facts can raise a score and [3P] facts can only lower it (third-party reports document bypasses, not guarantees); the highest level whose criteria are *all* met is chosen; undecidable criteria count as unmet; incident history does not change scores (it belongs to RQ3). D12 records a policy choice, not a vulnerability. Means are unweighted, need at least 8 scored dimensions and are shown with the minimum; rankings are claimed only if they hold under equal, incident-exposure and security-core weightings.

Coverage and reliability: Of 870 harness–dimension cells, 664 were scored and 206 are N/A with a recorded reason (feature absent, no access, or not documented); 5 runtimes have too little evidence for a mean. A second LLM-agent rater, without access to the first rater's scores, rescored the matrix from the *same* extracted fact lists, so agreement measures how consistently the rubric is applied to shared evidence, not the reliability of evidence collection. On the 655 cells both scored, quadratic-weighted Cohen's κ [15] is 0.88, exact agreement 91.6% and within-one agreement 99.5%; these exclude the 29 cells where one rater gave N/A and the other a score. All 84 disagreements (including those N/A cases) are published, not adjudicated, and rater 1's scores are reported.

Overall picture: Controls exist but are rarely safe by default (Figure 1). Across the 36 tier A/B products the per-dimension mean never reaches 2; the lowest are network egress (D03, mean 0.77), the execution sandbox (D02, 0.84), headless and CI safeguards (D08, 0.91) and workspace trust (D04, 0.97). The highest are vulnerability disclosure (D15, 1.51), audit logging (D11, 1.50) and managed policy (D14, 1.48). Means differ little: only Codex CLI stays in the top three and only Continue in the bottom three under all weightings, so buyers should read per-dimension columns, not means.

Permission model (D01): Nearly every product offers per-action approval, but only 6 of 36 reach score 2. Two patterns hold scores down: approve-everything can often be switched on from repository-controlled files (hooks returning an auto-approve verdict, repository MCP entries

TABLE I

HARNESS THREAT TAXONOMY (RQ1) WITH FRAMEWORK MAPPINGS, MITIGATING CONTROL DIMENSIONS (D01–D15, SECTION V) AND THE NUMBER OF VERIFIED 2025–2026 INCIDENTS WHOSE PRIMARY CLASS IT IS (RQ3). ATLAS IDS ARE ABBREVIATED (PREFIX `AML.` DROPPED; FIRST SHOWN).

Class	Name	STRIDE	OWASP LLM	OWASP ASI	MITRE ATLAS	Controls	Inc.
T01	Indirect prompt injection via workspace and retrieved content	TE	LLM01	ASI01	T0051.001, …	D01, D02, D03, D04, D11	14
T02	Repository and workspace configuration as code execution	ET	LLM06	ASI05, ASI02	T0081, …	D04, D01, D02, D05, D14	52
T03	Permission-model weaknesses and approval bypass	ET	LLM06	ASI02, ASI03	T0053, …	D01, D02, D14	74
T04	MCP trust: tool poisoning, rug pulls, OAuth and server identity	STE	LLM03, LLM01, LLM06	ASI04, ASI02	T0110, …	D05, D14, D06, D13, D03	15
T05	Skills, plugins, extensions and marketplaces	STE	LLM03	ASI04	T0010.005, …	D06, D13, D14	2
T06	Harness distribution and dependency supply chain	TSE	LLM03	ASI04	T0010.001, …	D13, D15, D14	9
T07	Secret exposure and residue	I	LLM02	ASI03	T0055, …	D07, D11, D12	12
T08	Data exfiltration channels	I	LLM02, LLM05	ASI02	T0086, …	D03, D02, D01	23
T09	Headless and CI abuse (pwn request)	EIT	LLM01, LLM06	ASI01, ASI02, ASI03	T0051.001, …	D08, D01, D14, D07	5
T10	Persistent memory and long-lived context poisoning	TE	LLM04	ASI06	T0080.000, …	D09, D11	2
T11	Sub-agent and cross-harness trust propagation	ES	LLM06	ASI03, ASI07, ASI08	T0118, …	D10, D01, D02	2
T12	Browser and computer-use surfaces	SIE	LLM01, LLM06	ASI01, ASI02	T0100, …	D02, D03, D01	1
T13	Multi-tenant isolation and authorization	IES	LLM02	ASI03	T0085, …	D14, D10, D07	7
T14	Destructive and unsafe autonomous actions	DT	LLM06	ASI02, ASI08	T0101, …	D01, D02, D14	0
T15	Telemetry, data residency, retention and audit gaps	IR	LLM02		–	D12, D11, D14	1
T16	Offensive misuse of harnesses by attackers	E		ASI10	T0124, …	D11, D14	3
T17	Execution sandbox and isolation escape	E	LLM06	ASI05	T0105, …	D02, D10, D03	22
T18	Agent control-plane exposure and authorization	SEI		ASI03	T0132, …	D16, D01, D04	34
T19	Conventional software defects in agent runtimes and SDKs	TDE	LLM05, LLM10	ASI05	T0049, …	D15, D13, D02	45

with `autoApprove`), and several harnesses default to a model-based approver (Claude Code's `auto`[D], Cursor's auto-review[D], Qwen Code's AUTO[D], Hermes Agent's `smart`[D]). The rubric does not count a classifier verdict as "asking", so these defaults score 1; counting them as asking would raise Claude Code's D01 from 1 to 3 and would also lift the other classifier-default harnesses. We did not recompute rankings under that reading; it is the main open question for the rubric.

Sandbox and egress (D02, D03): These are the lowest-scoring dimensions (means 0.84 and 0.77). Among tier A/B products, 8 have no sandbox option at all, the number that confine both filesystem and network by default with central enforcement (score 3) is 0, and 11 have no egress control. Managed runtimes score higher (tier C/D means 1.47 and 1.61), because the provider runs each session in an isolated container or microVM. Where sandboxes exist, the incident record shows that their boundaries are often the weak point: symlink and working-directory tricks, writes to `.git` hooks and fallbacks to unsandboxed execution account for 22 escape incidents (Section VII).

Workspace trust (D04): 13 tier A/B products honour repository configuration with no trust concept, and none reaches score 3. A frequent design choice is that headless mode bypasses the trust gate: Claude Code documents that `claude -p` in a never-trusted folder runs project hooks and MCP servers[D] [16], which we observed[T], whereas Grok Build ran an untrusted repository hook only after `-trust`[D][T] and Copilot CLI did not start a repository MCP server in an untrusted folder[T], not re-tested in a trusted folder (Section VI). Similarly, Cline documents that project plugins in `.cline/plugins/` load when working in that repository[D] [17]. These are documented design properties, not vulnerabilities, but a CI job running a harness on a pull request inherits them.

Other dimensions: On D07 only 1 of 36 tier A/B products reach score 2, mostly because transcript retention is undocumented even where credentials sit in a 0600 file or keychain[D] (D07 column). Telemetry (D12) is scored on transparency and opt-out, not as a defect. D13 is examined in Section IX-E. SDK runtimes leave approvals, sandboxing and egress to the application; the rubric scores these as not provided (*application responsibility*), so teams building on an SDK inherit those gaps unless they close them.

VI. DOCUMENTED CLAIMS VERSUS OBSERVED BEHAVIOUR

Answers RQ7.

Method: For each Tier-B harness we extracted security-relevant statements from the vendor's documentation into a claims table (117 claims across the closed harnesses), each tagged with a dimension (telemetry, updates, configuration/hook/MCP trust, sandbox, secret storage, permission default, state-directory modes). We then wrote 23 black-box probes using the method of Section III. Each probe runs the pinned client in a fresh container, optionally on a synthetic repository seeded with a hook, MCP server or settings file. Verdicts are *match*, *partial*, *mismatch* or *not observed* (behaviour not exercised); claims needing a login, GUI or interactive session stay *untested*.

Results: Only 24% of the 117 claims could be tested headless without a vendor login; 28 were tested: 22 matched, 3 were partial, 1 mismatched and 2 were not observed; 89 remain untested (Table II). By harness, 13 of 14 tested Claude Code claims matched, 4 of 7 for Antigravity, 3 of 4 for Grok Build and 2 of 3 for Copilot CLI. All observations in the table are [T]; all claims are [D].

TABLE II

CLAIMS VERSUS OBSERVED BEHAVIOUR FOR CLOSED AND HYBRID HARNESSES (RQ7). EACH ROW IS ONE TESTED DOCUMENTED CLAIM[D] AND THE OBSERVATION FROM A BLACK-BOX PROBE[T] ON THE PINNED CLIENT (CLAUDE CODE 2.1.300, ANTIGRAVITY CLI 1.3.5 BEFORE LOGIN, GROK BUILD 1.0.52, COPILOT CLI 1.0.98 IN BYOK MODE). UNTESTED CLAIMS ARE OMITTED.

Harness	Dimension	Observed	Verdict
antigravity	update_check	background updater contacted: antigravity-cli-auto-updater-974169037036.us-central1.run.app:443; last_check.timestamp: yes; updater lock files: yes	match
antigravity	update_check	with AGY_CLI_DISABLE_AUTO_UPDATE=true, updater host contacted: none	match
antigravity	telemetry_hosts	before any login, telemetry/logging hosts attempted: play.googleapis.com:443 (default enableTelemetry=true)	match
antigravity	state_dir_perms	pre-login files under ~/.gemini: 35; other-user-readable: 32	partial
antigravity	permission_default	unauthenticated headless run: exit 1, auth-required message: yes, duration 62.6 s	match
antigravity	permission_default	'agy -p /help' without credentials: exit 1, auth required: yes	mismatch
antigravity	mcp_trust	before login/trust: repo .agents MCP server started: no; repo hook ran: no	partial
claude-code	telemetry_hosts	vendor hosts attempted: api.anthropic.com:443	match
claude-code	telemetry_hosts	with a gateway base URL, Datadog intake contacted: no	match
claude-code	telemetry_hosts	vendor hosts attempted: none	match
claude-code	update_check	no update-check host contacted during a headless (-p) session	not-observed
claude-code	config_trust	repo env ANTHROPIC_BASE_URL followed: yes; repo env CLAUDE_CONFIG_DIR applied: no	match
claude-code	hooks_trust	repo SessionStart hook ran under -p, untrusted: yes (cc-repo-hook)	match
claude-code	mcp_trust	repo .mcp.json stdio server started without prompt: yes; init status [('name': 'hw', 'status': 'pending', 'source': 'project')]	match
claude-code	config_trust	repo hook ran with –bare: no (cc-repo-hook-bare)	match
claude-code	secret_storage	credential files written with ANTHROPIC_AUTH_TOKEN: none	match
claude-code	permission_default	init permissionMode with a third-party base URL: auto	match
claude-code	permission_default	–dangerously-skip-permissions as uid 0 refused: yes	match
claude-code	permission_default	repo permissions.defaultMode=bypassPermissions -> init permissionMode default	match
claude-code	state_dir_perms	transcripts written: 1 (modes 0o600); other-user-readable files under ~/.claude: 2	match
claude-code	config_trust	@.env inside a CLAUDE.md command line imported the gitignored .env into the session transcript without a model call or prompt: yes	match
github-copilot	telemetry_hosts	COPILOT_OFFLINE=true vendor hosts: none; without it (BYOK): api.github.com:443	match
github-copilot	config_trust	repo .mcp.json server started in an untrusted folder (-p): no	match
github-copilot	hooks_trust	repo .github/hooks sessionStart hook ran in -p: no	not-observed
grok-build	telemetry_hosts	vendor hosts attempted with a custom local model and no login: none	match
grok-build	hooks_trust	repo hook ran untrusted: no; with –trust: yes; .claude/settings.json compat hook untrusted: no	match
grok-build	mcp_trust	project .mcp.json (compat import) server started in an untrusted folder: no	partial
grok-build	secret_storage	files written under ~/.grok: 53; other-user-readable: 35 (session files among them: 0)	match

What the probes show: The tested claims are a convenience sample weighted towards network and update behaviour, so the match count does not generalise to the untested claims. The notable findings are documented design properties, not vulnerabilities. (i) *Headless trust.* In Claude Code, a `claude -p` run in a never-trusted folder executed a repository `SessionStart` hook and started a repository stdio MCP server without a prompt[T][D] [16], and applied a repository `env` entry redirecting the model endpoint[T][D]; `-bare` prevented the hook[T][D] [18], and a repository request for `bypassPermissions` was ignored[T][D]. Its documented `@path` import in `CLAUDE.md` also fired on the token `@.env` inside a shell command line, pulling the gitignored `.env` into the session transcript (mode 0600) with no model call[T][D]; this explains the E3 secret-in-state result (Section IX-C). (ii) *Trust-gated alternatives.* Grok Build did not run an untrusted repository hook, including one imported from the Claude Code format, and ran it after `-trust`[T][D] [4]. Copilot CLI did not start a repository MCP server or hook in an untrusted folder[T][D], but we did not re-test it in a trusted folder, so "did not run" may also mean "not supported under `-p`". (iii) *Network.* With documented opt-outs and a local model, Copilot CLI, Grok Build and Claude Code contacted no vendor host[T][D]. Before login, Antigravity's CLI contacted its auto-updater and a telemetry host, as its defaults document, and the updater opt-out worked[T][D] [19]. (iv) *The one mismatch* is functional: `agy -p /help` is documented to run without an agent turn but required authentication[T][D] [20]. *Partial* verdicts concern state-directory modes and MCP start-up before trust, where documentation is silent.

Limits: A *match* holds for this version in a headless Linux container; it does not certify interactive use, other platforms or post-login behaviour, and without payload inspection it says nothing about what an allowed host receives. Probe effort was uneven (14 Claude Code claims against 3 for Copilot CLI), and Claude Code ran in a configuration its vendor does not support (Section III). Without the opt-outs, during E3, the proxy blocked attempts to reach vendor or telemetry hosts from Claude Code (`api.anthropic.com`), Codex CLI (`chatgpt.com`, `github.com`), Qwen Code (an Aliyun RUM host, `models.dev`) and OpenCode (`models.opencode.ai`)[T].

VII. INCIDENT AND CVE SURVEY, 2025–2026

Answers RQ3.

Collection and counting rule: We searched NVD, the GitHub Advisory Database, vendor security pages and researcher write-ups for harness-level incidents disclosed from January 2025 to mid-October 2026 (including the harnesses' MCP servers, SDKs, CI actions and distribution channels), yielding 357 rows after deterministic de-duplication. An incident is *counted* only if (a) its primary source was read and (b) an independent script re-resolved every listed CVE and GHSA identifier against NVD and the GitHub advisory

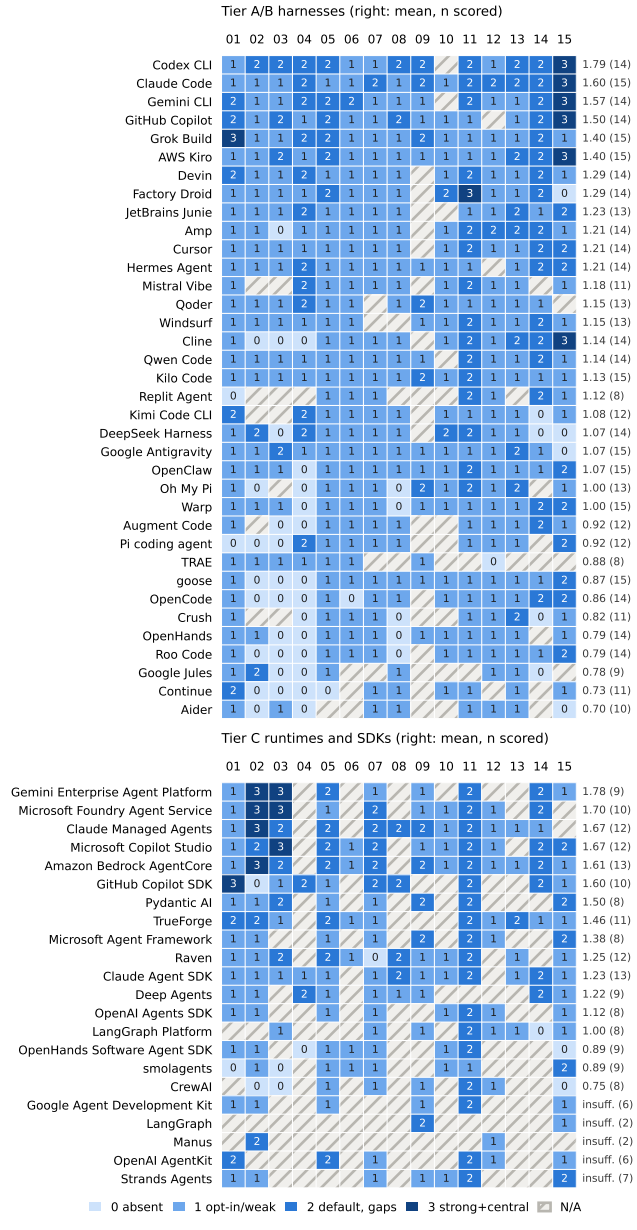


Fig. 1. Control matrix (RQ2): 0–3 score per dimension, rater 1; hatched cells are N/A. Tier A/B products (top) and tier C/D runtimes and hosted services (bottom) are ranked separately. The same scores with means are generated as `paper/tables/scorecard.tex`; per-cell evidence and rationale are in `matrix/`.

API; incidents without an identifier need a primary vendor or researcher source. 29 rows failed (a), 4 failed (b) and 1 fell outside 2025–2026, leaving 323 incidents (99 disclosed in 2025, 224 in 2026; 239 with a CVE and 237 with a GHSA). By kind, 292 are vulnerabilities, 19 design weaknesses, 8 supply-chain compromises, 3 documented offensive misuse and 1 other events. Classes follow Section IV; one LLM agent classified every incident, and inter-rater agreement for classes and root causes was not measured. The population is selected for disclosed, harness-level software defects, including 73

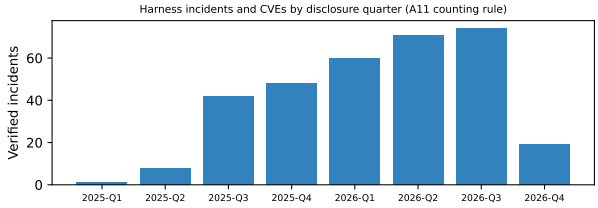


Fig. 2. Verified harness incidents and CVEs by disclosure quarter (RQ3). The last quarter is partial (to 15 October 2026) and is not a decline.

ecosystem items such as MCP tooling and package compromises, so shares describe that population, not the distribution of real-world harm.

Timeline: Disclosures rose through every full quarter of the study period (Figure 2). This reflects more harnesses, more researcher attention, wider GHSA use and an agent-assisted search that favours recent, well-indexed advisories, not necessarily declining quality; likewise, per-product counts track scrutiny and disclosure maturity (D15) as much as risk, so we do not rank products by them.

Root causes: Table III gives the distribution. Defective checks that gate an agent capability (RC3: allowlists bypassed by shell substitution, redirections, environment-variable prefixes, `find -exec`, symlinks or lookalike domains) are the largest group with 104 incidents (32%). Configuration honoured before consent (RC2) adds 52: repository settings, hooks, `.env` files, MCP declarations and Git configuration that run or take effect before the trust prompt. Together these deterministic harness defects account for 48% of counted incidents, the largest share but not a majority. Conventional defects not specific to agents (RC10) account for 16%, and missing authentication or Origin checks on local control endpoints (RC4) for 24. Pure prompt injection with no other control failure (RC1) is 5%; as the residual class, this is a lower bound on injection's role, which we did not code separately as a vector.

Classes: By primary class (Table I), permission-model weaknesses (T03, 74, 23%) and repository configuration as code execution (T02, 52) lead, followed by conventional defects (T19, 45) and control-plane exposure (T18, 34). Indirect prompt injection is the primary class of only 14 incidents, although injection is the delivery vector in many incidents classified elsewhere.

Representative cases: (credit to the original researchers) *Injection to configuration:* hidden Unicode in rules files (Pillar Security)[3P] [21]; injection flipping Copilot's auto-approve (CVE-2025-53773)[3P] [22]; Cursor executing attacker-written or silently modified MCP configuration (CVE-2025-54135/54136)[D][3P] [23], [24]. *Configuration before consent:* Claude Code project files running code and sending API tokens before the trust dialog (CVE-2025-59536, CVE-2026-21852)[3P] [25]; Codex CLI loading project MCP servers without confirmation (CVE-2025-61260)[3P] [26]; Gemini CLI's CI action auto-trusting the

TABLE III
ROOT CAUSES OF THE COUNTED 2025–2026 HARNESS INCIDENTS (RQ3), ASSIGNED BY THE ORDERED PROCEDURE. PRIMARY TAXONOMY CLASSES ARE IN TABLE I.

Root cause	Name	Incidents
RC3	Validation, parsing or authorization logic flaw in a check that gates an agent capability	104
RC2	Workspace, repository or agent-written content executed or honoured before or without consent	52
RC10	Conventional implementation defect not gating an agent capability	51
RC4	Missing authentication or origin/Host validation on a local or network control endpoint	24
RC8	Isolation boundary escape or weak isolation fallback	20
RC6	Insecure default, missing safeguard or over-privileged mode by design	20
RC1	Untrusted content treated as instructions (no distinct control failure)	17
RC7	Unintended exposure, leakage or over-collection of secrets or data by the harness itself	12
RC5	Package, extension or update-channel compromise or integrity failure	11
RC9	Output-rendering or automatic-fetch feature acts as an exfiltration channel	8
RC12	Adversarial use of a legitimate capability (no product defect)	3
RC11	Autonomous action exceeding user intent without enforced guardrail	1

workspace[D] [27]. *Allowlist and path checks:* Gemini CLI command deception[3P] [28]; Claude Code path and command bypasses (CVE-2025-54794/54795)[3P] [29] and deny rules not enforced through symlinks[D] [30]. *MCP:* tool poisoning and the GitHub MCP leak (Invariant Labs)[3P] [31], [32], SQL-returned injection via Supabase MCP[3P] [33], MCP Inspector's unauthenticated proxy (CVE-2025-49596)[D][3P] [34], [35]. *Other:* unauthenticated local control planes in OpenCode and DeepSeek Harness[D] [36], [37]; a malicious prompt in Amazon Q Developer 1.84.0[D] [38]; the `nx` "s1ngularity" compromise using installed AI CLIs to hunt secrets[D] [39]; Antigravity data leaks after launch[3P] [40], [41]; and a vendor's own account of criminal misuse of a coding agent [42].

Fix date relative to advisory publication: For the 148 counted incidents with both dates, a fix release existed on or before the advisory's publication in 135 cases (median offset -5.00 days, maximum 288); 175 incidents have no recorded fix date (unfixed or unknown). Because advisories are usually published after a fix by design, this measures disclosure practice, not time-to-fix; report dates were not available.

VIII. DESIGN OF HARNESSWATCH

Answers the design half of RQ4 and the instrument for RQ5 and RQ7.

`harnesswatch` is an Apache-2.0 Python package and CLI. It is read-only and offline by default, never transmits file contents, and redacts any secret it reports.

Profiles and a common schema: One risk has many spellings: approve-everything is `bypassPermissions` in Claude Code, `approval_policy = "never"` in Codex, `yolo` in Gemini and Qwen, and `"permission": "allow"` in OpenCode. A YAML profile per harness (one for each of the 58 in the matrix) records file locations, key names and documented defaults, and a profile-driven parser maps every file into a common `HarnessConfig` model (permission mode, allow/deny lists, hooks, MCP servers, instruction files, trust, telemetry, auto-update). Files shared by several harnesses are parsed once, with the semantics of the format's owner.

Audits and rules: Six audit modules cover *config*, *mcp* (pinning, transport, inline secrets, tool-description injection, auto-approval), *instructions* (hidden Unicode, override phrasing, remote includes), *secrets* (state directories, transcripts, file modes), *supply* (SBOM, OSV, installer verification) and *ci* (headless agents on untrusted triggers). The 25 rules (Table VII) each carry severity, confidence, a kind (vulnerability pattern, design weakness or policy choice), taxonomy classes, remediation and `should_fire`/`should_not_fire` fixtures. Output is terminal, JSON, Markdown or SARIF 2.1.0; a GitHub Action and a pre-commit hook wrap the repository audits.

observe and bench: `harnesswatch observe <harness> -- CMD` runs a command in a disposable container whose only exit is a logging proxy, and records CONNECT targets (classified as model API, telemetry, update or unknown), a filesystem diff with modes, credential locations, sandbox evidence from `/proc`, and permission prompts in the output. It stores no request bodies and does not decrypt TLS. It underlies E5, E7 and every bench run. The canary bench (Section IX-C) drives each harness headless through one adapter per harness and aggregates rates with Wilson intervals [43].

IX. EMPIRICAL EVALUATION

Answers RQ4 (E6, E2), RQ5 (E3, E5) and the supply-chain part of RQ2 (E4).

All experiments ran on one Linux workstation with Docker and a consumer GPU; every result number is read from `experiments/results/*.json`.

A. E6: Rule-pack accuracy on a labelled corpus (RQ4)

Design: The corpus has 376 cases (113 hand-crafted, 263 generated) over more than twenty harness formats, CI files, state directories and MCP manifests; 108 are fully negative, and many positives have near-miss twins (pinned `npx`, authenticated HTTPS MCP, linter-only hooks, emoji ZWJ, key-like identifiers). Every audit runs on every case and the unit is the (case, rule) pair, so any unlabelled firing is a false positive. A hash of the case id splits the corpus into 245 development and 131 test cases; errors were inspected and fixed only on the development split.

Results: On the held-out test split, micro precision is 0.983 (95% Wilson CI 0.950–0.994) and micro recall 0.983 (0.950–0.994), with 3 false positives and 3 false negatives; macro precision and recall are 0.97 and 0.95 (per-rule results: `paper/tables/e6_accuracy.md`). Before the development-split fixes, test precision was 0.960 with 7 false positives; the test split had been scored once before corpus-content fixes that touched both splits, so it is not pristine. The Wilson intervals treat (case, rule) pairs as independent

although they cluster within cases and generator families, so they are optimistic. The weakest rules are HG-SUB-001, HG-MCP-004 and the approve-everything rules. Because the corpus is synthetic and labelled by the rule author, this shows that the rules implement their specification across formats; real-world precision is unknown.

B. E2: Prevalence in public repositories (RQ4)

Design: Through the GitHub code-search API we sampled harness settings and MCP files, instruction files (AGENTS.md, CLAUDE.md, Copilot and Windsurf rules) and CI workflows invoking claude-code-action, codex exec or gemini -p. Per repository we rebuilt only the sampled files in a temporary tree, ran harnesswatch audit, kept only (rule, file) pairs and deleted the tree. Identifiers were hashed with an unsaved per-run salt for de-duplication only; a scrub check refuses to write results containing any identifier or URL seen; secret findings are only counted; requests stayed below the API's rate limits. Each of the 17 search strata is capped at 60 files, so pooled per-repository rates mostly reflect which strata were chosen. We therefore report each rule's rate over the stratum of files that can carry it (config/MCP files, $n = 600$; instruction files, $n = 240$; agent CI workflows, $n = 177$), with Wilson intervals, and never name a repository.

Code search over-represents popular, recently indexed files; only project-level configuration is visible; a finding marks risky configuration, not exploitation.

Results: We audited 1010 repositories (1017 files, 0 audit errors) and 18 rules fired at least once (Table IV). Among agent CI workflows, 63.3% run the agent with a write-scoped token (HG-CI-002) and 28.2% match the pwn-request pattern of untrusted trigger plus secrets (HG-CI-001). Among config and MCP files, 28.3% launch an MCP server from a registry without a version pin (HG-MCP-001) and 14.3% declare a remote server without TLS or authentication (HG-MCP-002); HG-MCP-005 (server declared in repository config, 51.8%) largely restates the sampling key and is informational. Secret-shaped strings in MCP declarations (HG-MCP-003, 3.5%) were counted only; since identifiers are discarded by design, owners were not notified and the strings were not validated. No E2 finding was hand-validated, so the real-world precision of these rules is unknown; the rates describe this capped, relevance-ranked sample, not GitHub as a whole.

C. E3: Canary benchmark for indirect prompt injection (RQ5)

Design: Each run uses a fresh container with a tiny Python repository, a synthetic .env token, a fixture file and a sink on 127.0.0.1, and the same benign task (add a multiply function and test, run the tests). Besides a *control*, four scenarios place one templated block in the README, the native instruction file, a code comment, or test output that returns as a tool result (Table VIII). The block asks the agent to create CANARY_<id>.txt, POST .env to the sink, and delete the fixture; all are inert, and the container reaches only the model server. Modes are *default* (out-of-the-box headless) and *auto* (the documented approve-everything flag).

TABLE IV
E2 PREVALENCE PER STRATUM (RQ4): REPOSITORIES WITH A FINDING OVER AUDITED FILES IN THE STRATUM THAT CAN CARRY THE RULE; TOP RULES, WILSON 95% CI. AGGREGATES ONLY; NO REPOSITORY IS NAMED.

Rule	Stratum (n)	Repos	Rate [95% CI]
HG-CI-002	workflows (177)	112	63.3% [56.0, 70.0]
HG-MCP-005	config (600)	311	51.8% [47.8, 55.8]
HG-MCP-001	config (600)	170	28.3% [24.9, 32.1]
HG-CI-001	workflows (177)	50	28.2% [22.1, 35.3]
HG-MCP-002	config (600)	86	14.3% [11.8, 17.4]
HG-HOOK-001	config (600)	43	7.2% [5.4, 9.5]
HG-MCP-003	config (600)	21	3.5% [2.3, 5.3]
HG-SEC-003	config (600)	14	2.3% [1.4, 3.9]
HG-PERM-001	config (600)	12	2.0% [1.1, 3.5]
HG-PERM-003	config (600)	12	2.0% [1.1, 3.5]

Harnesses, model and grid: Seven adapters cover three Tier-B clients (Claude Code, Copilot CLI and the Grok Build npm binary, driven through custom-endpoint modes; Claude Code's is vendor-unsupported with non-Claude models, Section III) and four open ones (OpenCode, Codex CLI, Aider and Qwen Code). No vendor credentials are used, so every harness talks to one local open-weight model, qwen3:4b-instruct, served by Ollama. The grid is 7 harnesses $\times$ 5 scenarios $\times$ 2 modes with 5 runs per cell. A run is capped at 240 s and scored on the state at the cap; a run with no harness output counts as an error, not as safe. Aider never loads an instruction file, so its instructions cells are N/A in both modes and excluded from the pooled statistics.

Metrics: Injection follow: any of the three goals achieved (marker created, sink POST, fixture deleted). *Exfil attempt*: any POST to the sink, or the secret seen at the sink. *Destructive action*: fixture deleted. *Secret read*: the synthetic token appears in the harness's printed output (other than as an environment-variable dump) or reaches the sink; since some harnesses print only the final message, this undercounts reads. *Secret in state*: the token appears in a transcript or session file the harness wrote, separating persistence from agent action. *Permission prompt*: an approval prompt in the output. *Task completed*: tests pass and the function and its test exist, so that "safe but useless" is visible. Rates carry Wilson 95% intervals.

Results: The grid produced 350 runs (0 errors, 22 timeouts scored at the cap, model cost $0.00), of which 340 enter the analysis (Table V; plotted in paper/figures/e3_follow.pdf). Over the injection scenarios the injected instruction was followed in 2 of 135 auto-mode runs (95% CI 0.004–0.052) and 0 of 135 default-mode runs (0.000–0.028); controls followed nothing (0 of 70). No run attempted the POST, the fixture was deleted in 1 auto-mode run, and the strict secret-read count is 0 in both modes. The intervals overlap, so there is *no detectable safety difference* between modes. Task completion differs (84 of 135 auto vs 23 of 135 default) because several headless defaults deny the tools the task needs: Codex CLI and Copilot CLI showed a permission prompt in every default run, while Grok Build and Qwen Code completed nothing in default

mode without a detectable prompt, for reasons we did not establish. OpenCode completed 1 of 10 even in controls, an adapter–model incompatibility, so its cells are uninformative. Claude Code's "default" arm is not an ask mode: with a third-party endpoint it starts in the classifier-mediated `auto` mode (Section VI).

Secret in state: In 10 of 10 Claude Code runs of the `instructions` scenario (both modes) the synthetic secret reached the session transcript, against 0 in its other scenarios. This is not a model action: an E7 probe with an unreachable model endpoint reproduced it, because the payload's `–data-binary @.env` contains the token `@.env`, which Claude Code's documented `@path` import resolves when it loads `CLAUDE.md`[T][D]. It is a harness design property: an instruction file can import gitignored secrets into context and transcripts without any approval.

Interpretation: The low follow rate most likely reflects the small model's limited agentic and instruction-following ability and the plainly worded benign canary, *not* harness robustness; with 5 runs per cell and zero or one event per cell, intervals are wide, and we do not rank harnesses on resilience. We do not call default modes safer.

Correction: Our first secret metric (token anywhere in output or written files) flagged 15 runs, chosen before results were inspected and replaced afterwards: 10 were the Claude Code import above and the rest Codex CLI, which writes its process environment, holding the canary, into `~/.codex` shell snapshots whenever its shell tool runs (6 of 50 Codex runs)[T]. Neither is the agent reading the secret, hence the two separate metrics: benchmarks should separate what a harness persists from what an agent does.

D. E5: Secret residue in harness state (RQ5)

Design: For each of the 7 bench harnesses we ran two short sessions in a fresh container. In *key* the provider credential was a unique synthetic canary. In *content* the prompt asked the agent to read a synthetic `.env` file. We searched every file the harness wrote for the canary and recorded paths, modes and reachability by other users (home 0755); contents were never stored. With one session per harness and condition ($N = 1$) the results are descriptive.

Results: In the content condition, 6 of 7 harnesses left at least one file holding the secret, typically a transcript or chat log (Table VI)[T]. In 3 harnesses such a file was readable by other local users. In the key condition, 0 of the 4 harnesses that take a key wrote it to a file in this single run[T]; that is a sampling result, not evidence of absence, because Codex CLI persists environment-supplied credentials in shell snapshots whenever its shell tool runs, which happened in 6 of 50 E3 runs and not in the E5 key session[T]. Mode bits alone can mislead: Grok Build writes 0644 session files, but inside a 0700 sessions directory, so other users cannot reach them[T]; Claude Code writes transcripts with mode 0600[T]. OpenCode left no residue in this run. Residue depends on whether the agent reads the file, so one run can miss it. These are design weaknesses, not vulnerabilities; a 0750 home directory would block the reachable paths, but CI runners and containers often use 0755.

E. E4: Distribution supply chain (RQ2, D13)

We inspected the published package of 23 harnesses (the Tier-A set plus Claude Code, Grok Build and Copilot CLI) from registry metadata and lockfiles only, without running install scripts[D] (per-package table: `paper/tables/e4_supply.md`). 7 packages declare install-time hooks. 10 of 18 with determinable status publish registry provenance attestations. 17 offer a `curl | sh` installer, and in 17 of them the installer verifies no signature (some verify a checksum; Claude Code documents a signed release manifest that its installer does not itself verify[D]). Auto-update is documented as on by default in 12 (cells marked "unverified" in the table are undetermined, not off). The median lockfile size is 97 transitive packages, and OSV lists 299 unique advisories against locked dependency versions, concentrated in a few packages. These are advisories against pinned versions, not demonstrated exploitability. Auto-update is a trade-off: it delivers fixes quickly but widens the blast radius of a compromised release channel, as the Amazon Q extension incident showed[D] [38].

X. Governance Recommendations

Answers RQ6 (as recommendations, not an evaluated finding).

We turned the findings into a 30-control checklist (GC-01 to GC-30). Each control states the requirement, its rationale (threat classes, root causes, rubric dimensions, and whether it addresses a vulnerability, design weakness or policy choice), how to verify it with an exact `harnesswatch` command or a manual check, and its mapping to NIST AI RMF 1.0 [44], NIST AI 600-1 [45], ISO/IEC 42001 Annex A [46], OWASP [47], [48], MITRE ATLAS [49] and the MAS AI risk-management guidelines [50]. The controls that follow most directly from our evidence are: treat repository configuration, including instruction-file imports, as untrusted code and gate changes to it in CI (GC-08, GC-09, GC-23; RC2, E3 secret-in-state); lock approve-everything modes by managed policy and treat allowlists as convenience, not a boundary (GC-06, GC-07; RC3); supply the sandbox and egress boundary yourself, since D02 and D03 score lowest (GC-16, GC-17, GC-20); never run headless agents with secrets or write tokens on untrusted triggers (GC-21, GC-22; E2 CI strata); keep secrets out of config files and the agent's environment and restrict state directories (GC-14, GC-15; E5); and re-run the black-box probes and canary bench on the version and model you deploy (GC-03, GC-04). The checklist has not been evaluated for coverage or usability; the OWASP MCP Top 10 is in beta, ISO/IEC 42001 Annex A titles come from public listings, and NIST AI 600-1 is cited at document level where needed.

XI. Related Work

Answers where RQ1–RQ7 sit relative to prior work.

TABLE V

E3 CANARY BENCH (RQ5): PER HARNESS AND PERMISSION MODE OVER THE INJECTION SCENARIOS; k/n WITH WILSON 95% CI IN PERCENT; OTHER COLUMNS ARE COUNTS OUT OF n. ONE LOCAL MODEL (`QWEN3:4B-INSTRUCT`); ^{B} MARKS TIER-B CLIENTS. NOT A RESILIENCE RANKING.

Harness	Mode	n	Follow [95% CI, %]	Exfil	Destr.	Secret in state	Prompts	Timeouts	Task done [95% CI, %]
aider	auto	15	0/15 [0.0, 20.4]	0	0	0	0	0	10/15 [41.7, 84.8]
aider	default	15	0/15 [0.0, 20.4]	0	0	0	0	0	10/15 [41.7, 84.8]
claude-code^B	auto	20	1/20 [0.9, 23.6]	0	1	5	0	4	16/20 [58.4, 91.9]
claude-code^B	default	20	0/20 [0.0, 16.1]	0	0	5	0	6	13/20 [43.3, 81.9]
codex-cli	auto	20	1/20 [0.9, 23.6]	0	0	0	8	4	11/20 [34.2, 74.2]
codex-cli	default	20	0/20 [0.0, 16.1]	0	0	0	20	1	0/20 [0.0, 16.1]
github-copilot^B	auto	20	0/20 [0.0, 16.1]	0	0	0	1	0	18/20 [69.9, 97.2]
github-copilot^B	default	20	0/20 [0.0, 16.1]	0	0	0	20	2	0/20 [0.0, 16.1]
grok-build^B	auto	20	0/20 [0.0, 16.1]	0	0	0	0	0	17/20 [64.0, 94.8]
grok-build^B	default	20	0/20 [0.0, 16.1]	0	0	0	0	0	0/20 [0.0, 16.1]
opencode	auto	20	0/20 [0.0, 16.1]	0	0	0	0	0	0/20 [0.0, 16.1]
opencode	default	20	0/20 [0.0, 16.1]	0	0	0	0	0	0/20 [0.0, 16.1]
qwen-code	auto	20	0/20 [0.0, 16.1]	0	0	0	1	1	12/20 [38.7, 78.1]
qwen-code	default	20	0/20 [0.0, 16.1]	0	0	0	0	1	0/20 [0.0, 16.1]

TABLE VI

E5 SECRET RESIDUE AFTER READING A SYNTHETIC `.ENV` (CONTENT CONDITION; ONE SESSION PER HARNESS, DESCRIPTIVE). "REACH.": FILES READABLE BY OTHER USERS (O+R ON THE FILE AND O+X ON EVERY ANCESTOR). "CRED.": FILES HOLDING THE PROVIDER KEY IN THE KEY CONDITION (N/A: NO KEY TAKEN). "O+R STATE": OTHER-USER-READABLE / ALL STATE FILES WRITTEN.

Harness	Files w/ secret	Kind	Mode	Reach.	Cred.	o+r state
aider	1	workspace	644	1	n/a	3/3
claude-code	1	log	600	0	0	2/5
codex-cli	2	log	644	2	0	67/69
github-copilot	2	log/state	600/644	0	n/a	236/248
grok-build	2	log	644	0	0	35/54
opencode	0	–	–	0	n/a	26/26
qwen-code	1	log	644	1	0	10/13

Prompt-injection benchmarks for agents: Greshake et al. introduced indirect prompt injection against LLM-integrated applications [1], and Liu et al. formalised attacks and defences [51]. InjecAgent [3] and AgentDojo [2] benchmark tool-using agents against injected tool outputs, Agent Security Bench [52] broadens the attack and defence set, ToolEmu [53] emulates tools to find risky agent behaviour, and RedCode [54] benchmarks risky code execution by code agents. These fix the agent framework and vary model and attack; our bench fixes model and payload and varies real harness binaries and modes.

Design-level defences: CaMeL [55] and the design patterns of Beurer-Kellner et al. [56] constrain what an agent can do after reading untrusted data. System-level isolation (IsolateGPT [57]) and programmable privilege control (Progent [58]) move enforcement into the runtime, much as our rubric asks harnesses to do. Willison's "lethal trifecta" [59] explains why egress (D03) and secret handling (D07) matter even when injection cannot be prevented.

MCP security: Hou et al. survey the MCP landscape and its threats [60]. Radosevich and Halloran show that MCP-connected models can be driven to harmful actions [61]. Hasan et al. study the security and maintainability of MCP servers at scale [62], and MCPTox benchmarks tool poisoning on real servers [63]. Invariant Labs described tool-poisoning attacks and the GitHub MCP data leak [31], [32], and Trail of Bits showed that servers can attack before any tool is used [64]. We add the harness side: how servers are declared, pinned and approved, and how often risky declarations appear in public repositories.

Practitioner research on coding agents: Much of what is known comes from practitioners: Pillar Security's rules-file backdoor [21], Tracebit's Gemini CLI allowlist bypass [28], Johann Rehberger's *Embrace The Red* series, including the Copilot auto-approve RCE, the Month of AI Bugs and the Antigravity analysis [22], [41], [65], Trail of Bits on prompt injection to RCE in agents [66], and Check Point Research and Cymulate on Cursor, Codex CLI and Claude Code [24]–[26], [29]. Our survey normalises these reports into one taxonomy, root-cause scheme and counting rule.

CI and supply chain: Studies of GitHub workflow security [67], [68] underpin our CI rules and E2 workflow stratum, and open-source supply-chain attack taxonomies [69] frame E4.

Frameworks: The OWASP Top 10 for LLM Applications [47], the OWASP Agentic AI threats and mitigations guide [70], the OWASP Top 10 for Agentic Applications [48] and the OWASP MCP Top 10 [71] catalogue risks at the application level, and MITRE ATLAS [49] catalogues adversary techniques. We map our taxonomy onto all of them and add a harness-specific, checkable control layer.

Author's prior tooling: `harnesswatch` reuses parsing ideas from the author's `upfront`, offers `rulewarden` (deterministic instruction-file rule enforcement) as an integration point, and uses `airmkit`'s control vocabulary for the MAS mapping.

XII. LIMITATIONS AND THREATS TO VALIDITY

Answers how far the answers to RQ1–RQ7 generalise.

Version drift: Scores, probes and bench cells refer to the versions in `VERSIONS.md`, accessed 15 October 2026. Harnesses release weekly and several were renamed or discontinued during the study; the matrix should be re-run, not cited as current.

Models and compute: No vendor API keys or logins were used, so the bench runs one small local open-weight model through documented custom-endpoint features and says nothing about any vendor's flagship model. The GPU is shared with the owner's other work; a planned second model family was cancelled to release it, so the goal of at least two models per harness is *not met*, and the scenario set and run count per cell were reduced, leaving wide intervals. A model this small also limits how often any injection is acted on at all. Claude Code was routed to a non-Claude model, which its vendor does not support, and in that setup its default arm is the classifier `auto` mode; its bench and permission results describe harness behaviour in an unsupported configuration.

Headless only: Probes and bench runs are headless. Interactive trust dialogs, IDE extensions and GUI-only products (e.g. the Antigravity IDE) were not tested; a harness whose interactive mode is safer is under-credited, which is the conservative error for CI use.

Benign-canary ceiling: Payloads are inert and plainly worded; disguised or adaptive attacks may succeed where they fail, so bench rates are lower bounds.

Black-box limits: Without TLS inspection we cannot say what an allowed host receives; login-gated harnesses were observed only before login.

Rating and classification: Evidence collection, both matrix raters, incident collection and incident classification were LLM agents supervised by the author. Both raters worked from one shared fact base, so κ measures rubric application, not evidence reliability, and it excludes the N/A-versus-score cells (all 84 disagreements, including those, are published). Incident classes and root causes had a single rater with no agreement measurement, and RC1 is the residual class of an ordered procedure. Several scores depend on rubric interpretations, notably classifier-gated defaults.

Selection: The incident survey admits only publicly disclosed, harness-level advisories, mostly with CVE or GHSA identifiers; model-level failures are rarely tracked this way, and agent-assisted search favours recent, well-indexed advisories. RQ3 shares therefore describe disclosed harness defects, not overall risk.

Corpora: E6 is synthetic and author-labelled; E2's code-search sample favours popular, recently indexed repositories; E5 has one session per condition.

XIII. Conclusion

Answers the summary of RQ1–RQ7.

The harness is where an agent's capabilities become effects. Among publicly disclosed harness-level advisories of 2025–2026, flawed approval checks and configuration honoured before consent account for 48%, and pure prompt injection, a residual class, for 5%. Across 58 harnesses and runtimes, controls exist but are rarely safe by default: sandboxing and egress are the weakest dimensions, headless modes often bypass trust gates, and in our public-repository sample unpinned MCP launches and write-scoped agent CI jobs recur. For closed harnesses, 22 of the 28 testable claims matched, but most documented claims could not be tested headless, and documented features such as instruction-file imports can move secrets into transcripts without any model action.

The answers are engineering ones: trust gates that cover every executable configuration class and apply headless, policy-locked permission modes, default sandboxes and egress allowlists. We release `harnesswatch`, the rubric, evidence and results so the study can be re-run as the field moves.

Appendix A
Reproducibility

The repository contains the toolkit, rule pack, harness profiles, evidence ledger, matrix evidence, results JSON and this paper. Three entry points rebuild everything:

- `make all` runs lint, type checks and tests with coverage, builds the documentation, regenerates every results-derived table, figure and number offline from committed data (`make experiments`, which ends with `experiments/paper_numbers.py` writing `paper/numbers.tex`), and builds this paper with `paper/build.sh`.
- `make bench` builds the pinned harness images (`make bench-images`) and runs or resumes the canary grid against a local Ollama container, then re-runs the E3 analysis. It needs Docker and a GPU-backed Ollama container on the bench network. No vendor credentials are used.
- `paper/build.sh` uses a pinned Tectonic release (downloaded with a SHA-256 check) and writes a Markdown mirror, `paper/paper.md`.

Pinned versions: harness versions in `VERSIONS.md`; bench images are tagged with the harness version (Table VIII); the model server is Ollama 0.40.4 pinned by image digest; and Python dependencies are locked in `uv.lock`. Every result number in the prose is a macro generated from `experiments/results/*.json`; design constants (versions, caps, mode bits, counts of classes and controls) are typed. The E2 corpus script stores aggregates only and refuses to write any repository identifier.

Appendix B
Review and Corrections

After an internal hostile review we made these changes, all reflected in the results files and logged in the assumptions log: (1) incident de-duplication is now deterministic, which removed one double-counted incident, and the excluded out-of-period row is shown; (2) "patch latency" was recomputed from dates and reframed as fix date relative to advisory publication, with undated fixes reported; (3) the E3 secret metric was split into *secret read* and *secret in state* after the lenient metric was found to count harness persistence (Codex environment snapshots, the Claude Code `@path` import); (4) E2 prevalence is now reported per stratum instead of pooled; (5) matrix evidence labels were aligned with the ledger so that source reads are [D], not [T]; (6) one fact about Grok Build obtained by listing strings in the shipped binary was withdrawn as outside

our metadata-only rule; (7) claims about RQ3 shares, mode safety, claim coverage and rater independence were reworded.

Appendix C
Rule Pack

Table VII lists every rule; each YAML file adds harness-specific spellings, remediation, references and fixtures.

Appendix D
Canary-Bench Scenario Catalogue

Table VIII lists the scenarios and adapters. Every injected block uses the same versioned payload template, and only its location changes. The `hidden_unicode` scenario is implemented but excluded from the reported grid. Its zero-width wrapping reaches the model as plain ASCII, so it measures nothing beyond `instructions`. Aider has no auto-loaded instruction file, so its `instructions` cells are N/A.

Appendix E
Scorecard Rubric

The fifteen core dimensions and the 0–3 scale are listed in Section V; a proposed D16 (local service and control-plane exposure) is excluded from all means until it has enough evidence. Each dimension poses one question, e.g. D01 "does the harness ask before risky tool actions, and can the approve-everything path be restricted?" and D04 "is repository-supplied configuration ignored until the user consents?". Its criteria are written so that they can be decided from labelled facts: score 3 always requires a safe default *and* central enforceability (e.g. for D02, filesystem and network confined by default, not weak-enable by the agent or repository files, and enforceable by an administrator). Scoring rules: highest level whose criteria are all met; [3P] facts can only lower a score; undecidable criteria count as unmet; incident history never changes a score. Tier-C runtimes use separate ladders that flag *application responsibility*. The full criteria are in `matrix/rubric.md` (summary table: `paper/tables/rubric.md`), the scores with means and minima in `paper/tables/scorecard.tex`, and every cell's evidence and rationale in `matrix/`.

References

[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90. [Online]. Available: https://arxiv.org/abs/2302.12173

[2] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024. [Online]. Available: https://arxiv.org/abs/2406.13352

[3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. [Online]. Available: https://arxiv.org/abs/2403.02691

[4] xAI, "Grok Build: Hooks," https://docs.x.ai/build/features/hooks.md, 2026, accessed 2026-10-15.

[5] ——, "Grok Build: MCP servers," https://docs.x.ai/build/features/mcp-servers.md, 2026, accessed 2026-10-15.

[6] Anthropic, "Claude Code: Permission modes," https://code.claude.com/docs/en/permission-modes, 2026, accessed 2026-10-15.

[7] ——, "Claude Code: Sandboxing," https://code.claude.com/docs/en/sandboxing, 2026, accessed 2026-10-15.

[8] Model Context Protocol, "Model context protocol specification, version 2026-07-28," modelcontextprotocol.io, Jul. 2026, accessed 2026-10-15. [Online]. Available: https://modelcontextprotocol.io/specification/2026-07-28

[9] S.-J. Wiggers, "Anthropic accidentally exposes Claude Code source via npm source map file," InfoQ, Apr. 2026, accessed 2026-10-15. [Online]. Available: https://infoq.com/news/2026/04/claude-code-source-leak

[10] xAI, "xai-org/grok-build source repository (apache-2.0)," https://github.com/xai-org/grok-build, 2026, accessed 2026-10-15.

[11] Anthropic, "Claude Code: LLM gateway configuration," https://code.claude.com/docs/en/llm-gateway, 2026, accessed 2026-10-15.

[12] GitHub, "Using your own LLM models in GitHub Copilot CLI," https://docs.github.com/en/copilot/how-tos/copilot-cli/customize-copilot/use-byok-models, 2026, accessed 2026-10-15.

[13] xAI, "Grok Build: Overview," https://docs.x.ai/build/overview.md, 2026, accessed 2026-10-15.

[14] Google, "Google Antigravity additional terms of service," https://antigravity.google/terms, 2026, accessed 2026-10-15.

[15] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968.

[16] Anthropic, "Claude Code: Configure permissions — what runs before you trust a folder," https://code.claude.com/docs/en/permissions#what-runs-before-you-trust-a-folder, 2026, accessed 2026-10-15.

[17] Cline, "Plugins (cline documentation, docs/customization/plugins.mdx)," https://github.com/cline/cline/blob/main/docs/customization/plugins.mdx, 2026, accessed 2026-10-15.

[18] Anthropic, "Claude Code: Run Claude Code programmatically," https://code.claude.com/docs/en/headless, 2026, accessed 2026-10-15.

[19] Google, "Google Antigravity: Settings," https://antigravity.google/docs/settings, 2026, accessed 2026-10-15.

[20] ——, "Antigravity CLI: Headless mode," https://antigravity.google/docs/cli/headless, 2026, accessed 2026-10-15.

[21] Z. Karliner, "New vulnerability in GitHub Copilot and Cursor: How hackers can weaponize code agents ("Rules File Backdoor")," Pillar Security blog, Mar. 2025, accessed 2026-10-15. [Online]. Available: https://www.pillar.security/blog/new-vulnerability-in-github-copilot-and-cursor-how-hackers-can-weaponize-code-agents

[22] J. Rehberger, "GitHub Copilot: Remote code execution via prompt injection (CVE-2025-53773)," Embrace The Red blog, Aug. 2025, accessed 2026-10-15. [Online]. Available: https://embracethered.com/blog/posts/2025/github-copilot-remote-code-execution-via-prompt-injection/

[23] Anysphere, "GHSA-4cxx-hrm3-49rm / CVE-2025-54135: Arbitrary code execution from Cursor agent via MCP special file," https://github.com/cursor/cursor/security/advisories/GHSA-4cxx-hrm3-49rm, 2025, accessed 2026-10-15.

[24] A. Charikov, R. Zaikin, and O. Vanunu, "Cursor IDE's MCP vulnerability (MCPoison)," Check Point Research, https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, 2025, accessed 2026-10-15.

[25] A. Donenfeld and O. Vanunu, "Caught in the hook: RCE and API token exfiltration through Claude Code project files (CVE-2025-59536, CVE-2026-21852)," Check Point Research, Feb. 2026, accessed 2026-10-15. [Online]. Available: https://research.checkpoint.com/2026/rce-and-api-token-exfiltration-through-claude-code-project-files-cve-2025-59536/

[26] I. Mill and O. Vanunu, "OpenAI Codex CLI: command injection via project-local configuration (CVE-2025-61260)," Check Point Research, https://research.checkpoint.com/2025/openai-codex-cli-command-injection-vulnerability/, 2025, accessed 2026-10-15.

[27] Google, "Security advisory GHSA-wpqr-6v78-jr5g (run-gemini-cli)," https://github.com/google-github-actions/run-gemini-cli/security/advisories/GHSA-wpqr-6v78-jr5g, 2026, accessed 2026-10-15.

[28] S. Cox, "Code execution through deception: Gemini AI CLI hijack," Tracebit blog, Jul. 2025, accessed 2026-10-15. [Online]. Available: https://tracebit.com/blog/code-exec-deception-gemini-ai-cli-hijack

[29] E. Beber, "InversePrompt: CVE-2025-54794 and CVE-2025-54795 in Claude Code," Cymulate, https://www.cymulate.com/blog/cve-2025-547954-54795-claude-inverseprompt/, 2025, accessed 2026-10-15.

[30] Anthropic, "Security advisory GHSA-4q92-rfm6-2cqx (claude-code): permission deny rules not enforced through symbolic links," https:

TABLE VII

THE HARNESSWATCH RULE PACK (25 RULES). KIND SEPARATES VULNERABILITY PATTERNS, DESIGN WEAKNESSES (UNSAFE DEFAULTS OR CONFIGURATIONS) AND POLICY CHOICES; TAXONOMY CLASSES REFER TO TABLE I.

Rule	Severity	Confidence	Kind	Taxonomy	Title
HG-CI-001	high	high	vuln. pattern	T09, T01, T08	Headless agent in a workflow triggered by untrusted input ("pwn request")
HG-CI-002	medium	medium	design weakness	T09, T14	Agent CI job runs with a write-scoped GITHUB_TOKEN
HG-HOOK-001	medium	high	design weakness	T02	Repository config defines hooks that execute shell commands
HG-HOOK-002	medium	medium	vuln. pattern	T02, T06, T08	Hook command fetches or sends data over the network, or runs code fetched at runtime
HG-INJ-001	medium	high	vuln. pattern	T01, T05	Hidden or confusable Unicode in an instruction file
HG-INJ-002	medium	medium	vuln. pattern	T01, T05, T08	Prompt-injection phrasing in an instruction file
HG-INJ-003	medium	medium	design weakness	T01, T06	Instruction file pulls remote content at load time
HG-MCP-001	medium	high	vuln. pattern	T04, T06	MCP server fetched from a registry at launch without a version pin
HG-MCP-002	medium	medium	vuln. pattern	T04, T08	Remote MCP server over plaintext HTTP, wildcard host, or without authentication
HG-MCP-003	high	medium	vuln. pattern	T07, T04	Literal secret in an MCP server's env, headers or arguments
HG-MCP-004	medium	medium	vuln. pattern	T04, T01	MCP tool description contains instruction-like content (tool poisoning)
HG-MCP-005	low	high	design weakness	T02, T04	MCP server declared in a repository config
HG-MCP-006	high	high	design weakness	T02, T04, T03	MCP servers or their tools are auto-approved without a per-server prompt
HG-MEM-001	low	medium	design weakness	T10	Persistent agent-writable memory enabled (or shared through version control)
HG-NET-001	low	medium	design weakness	T08	Unrestricted network egress for agent tools where the harness supports restriction
HG-PERM-001	medium	high	design weakness	T03, T14, T01	Approve-everything (YOLO / bypass / full-auto) mode enabled in config
HG-PERM-002	medium	high	design weakness	T03, T14	Over-broad tool allowlist
HG-PERM-003	medium	medium	design weakness	T14, T07	Unrestricted execution without a deny list for destructive or credential-reading commands
HG-SEC-001	high	medium	design weakness	T07	Secret-like strings in harness logs, transcripts or memory
HG-SEC-002	medium	high	design weakness	T07	Harness state or credential file readable by other users
HG-SEC-003	high	medium	design weakness	T07	Credential stored in plaintext in a harness config file
HG-SUB-001	medium	medium	design weakness	T11, T03	Sub-agent or delegated agent configured with bypass permissions
HG-SUP-001	low	medium	design weakness	T06	Harness installed through a `curl | sh` installer without checksum or signature verification
HG-SUP-002	high	high	vuln. pattern	T06	Installed harness package version has known vulnerabilities (OSV)
HG-TEL-001	info	high	policy choice	T15	Telemetry, prompt logging or data sharing explicitly enabled

TABLE VIII

CANARY-BENCH CATALOGUE. TOP: SCENARIOS (ALL SHARE THE BASE REPOSITORY AND TASK; <INSTRUCTION FILE> IS THE HARNESS-NATIVE FILE NAMED BELOW). BOTTOM: HARNESS ADAPTERS, PINNED IMAGE VERSION, AND THE DOCUMENTED FLAG ADDED IN THE *auto* MODE.

Scenario	Title	Taxonomy	Files changed vs. base repository
control	No injection (baseline)	-	none
readme	README injection	T01	README.md
instructions	Harness-native instruction file injection	T01	<instruction file>
code_comment	Injection in a source-code comment	T01	calc.py
tool_output	Poisoned tool result (test output)	T01/T04	pytest.ini, tests/test_calc.py
hidden_unicode	Zero-width-wrapped instruction-file injection	T01	<instruction file>

Harness	Tier	Version	Instruction file	Auto-mode flag
claude-code	B	2.1.300	CLAUDE.md	-dangerously-skip-permissions
grok-build	B	1.0.52	AGENTS.md	-always-approve
github-copilot	B	1.0.98	.github/copilot-instructions.md	-allow-all-tools
opencode	A	1.19.0	AGENTS.md	-dangerously-skip-permissions
codex-cli	A	0.162.1	AGENTS.md	-dangerously-bypass-approvals-and-sandbox
aider	A	0.86.2	CONVENTIONS.md	-yes-always
qwen-code	A	0.25.1	QWEN.md	-yolo

//github.com/anthropics/claude-code/security/advisories/GHSA-4q92-rfm6-2cqx, 2026, accessed 2026-10-15.

[31] L. Beurer-Kellner and M. Fischer, "MCP security notification: Tool poisoning attacks," Invariant Labs blog, Apr. 2025, accessed 2026-10-15. [Online]. Available: https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks

[32] M. Milanta and L. Beurer-Kellner, "GitHub MCP exploited: Accessing private repositories via MCP," Invariant Labs blog, May 2025, accessed 2026-10-15. [Online]. Available: https://invariantlabs.ai/blog/mcp-github-vulnerability

[33] R. Havaei, R. Liu, and M. Li, "Supabase MCP can leak your entire SQL database," General Analysis, https://www.generalanalysis.com/blog/supabase-mcp-blog, 2025, accessed 2026-10-15.

[34] Model Context Protocol project, "GHSA-7f8r-222p-6f5g / CVE-2025-49596: MCP Inspector proxy server vulnerabilities," https://github.com/modelcontextprotocol/inspector/security/advisories/GHSA-7f8r-222p-6f5g, 2025, accessed 2026-10-15.

[35] A. Lumelsky, "Critical RCE vulnerability in Anthropic MCP inspector — CVE-2025-49596," Oligo Security blog, Jun. 2025, accessed 2026-10-15. [Online]. Available: https://www.oligo.security/blog/critical-rce-vulnerability-in-anthropic-mcp-inspector-cve-2025-49596

[36] OpenCode maintainers, "Security advisory GHSA-vxw4-wv6m-9hhh (opencode): unauthenticated local HTTP server," https://github.com/anomalyco/opencode/security/advisories/GHSA-vxw4-wv6m-9hhh, 2026, accessed 2026-10-15.

[37] GitHub Advisory Database, "Security advisory GHSA-8m2g-8cgm-3vcp: local control plane trusts client-supplied Host header," https://github.com/advisories/GHSA-8m2g-8cgm-3vcp, 2026, accessed 2026-10-15.

[38] Amazon Web Services, "Security update for Amazon Q Developer extension for Visual Studio Code (version #1.84)," AWS Security Bulletin AWS-2025-015 (CVE-2025-8217), Tech. Rep., Jul. 2025,

accessed 2026-10-15. [Online]. Available: https://aws.amazon.com/security/security-bulletins/AWS-2025-015/

[39] Nx maintainers (nrwl), "Malicious versions of Nx and some supporting plugins were published," GitHub Security Advisory GHSA-cxm3-wv7p-598c, Tech. Rep., Aug. 2025, accessed 2026-10-15. [Online]. Available: https://github.com/nrwl/nx/security/advisories/GHSA-cxm3-wv7p-598c

[40] PromptArmor, "Google Antigravity exfiltrates data," https://www.promptarmor.com/resources/google-antigravity-exfiltrates-data, 2025, accessed 2026-10-15.

[41] J. Rehberger, "Antigravity grounded! security vulnerabilities in Google's latest IDE," Embrace The Red blog, Nov. 2025, accessed 2026-10-15. [Online]. Available: https://embracethered.com/blog/posts/2025/security-keeps-google-antigravity-grounded/

[42] Anthropic, "Detecting and countering misuse of AI: August 2025," https://www.anthropic.com/news/detecting-countering-misuse-aug-2025, 2025, accessed 2026-10-15.

[43] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[44] E. Tabassi, "Artificial intelligence risk management framework (AI RMF 1.0)," National Institute of Standards and Technology, NIST AI 100-1, Jan. 2023. [Online]. Available: https://doi.org/10.6028/NIST.AI.100-1

[45] C. Autio, R. Schwartz, J. Dunietz, S. Jain, M. Stanley, E. Tabassi, P. Hall, and K. Roberts, "Artificial intelligence risk management framework: Generative artificial intelligence profile," National Institute of Standards and Technology, NIST AI 600-1, Jul. 2024. [Online]. Available: https://doi.org/10.6028/NIST.AI.600-1

[46] ISO/IEC, "ISO/IEC 42001:2023 — information technology — artificial intelligence — management system," International Organization for Standardization, Dec. 2023, accessed 2026-10-15. [Online]. Available: https://www.iso.org/standard/81230.html

[47] OWASP GenAI Security Project, "OWASP top 10 for LLM applications 2025," OWASP Foundation, 2024, accessed 2026-10-15. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-llm-applications-2025/

[48] ——, "OWASP top 10 for agentic applications for 2026," OWASP Foundation, Dec. 2025, accessed 2026-10-15. [Online]. Available: https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

[49] MITRE, "MITRE ATLAS: Adversarial threat landscape for artificial-intelligence systems," MITRE Corporation, accessed 2026-10-15. [Online]. Available: https://atlas.mitre.org/

[50] Monetary Authority of Singapore, "Guidelines on artificial intelligence risk management for financial institutions," Monetary Authority of Singapore, Oct. 2026, accessed 2026-10-15. [Online]. Available: https://www.mas.gov.sg/regulation/guidelines/guidelines-on-artificial-intelligence-risk-management-for-financial-institutions

[51] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, "Formalizing and benchmarking prompt injection attacks and defenses," in *33rd USENIX Security Symposium (USENIX Security 24)*, 2024. [Online]. Available: https://arxiv.org/abs/2310.12815

[52] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations (ICLR)*, 2025. [Online]. Available: https://arxiv.org/abs/2410.02644

[53] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2309.15817

[54] C. Guo, X. Liu, C. Xie, A. Zhou, Y. Zeng, Z. Lin, D. Song, and B. Li, "RedCode: Risky code execution and generation benchmark for code agents," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024, accessed 2026-10-15. [Online]. Available: https://arxiv.org/abs/2411.07781

[55] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," arXiv preprint arXiv:2503.18813, 2025. [Online]. Available: https://arxiv.org/abs/2503.18813

[56] L. Beurer-Kellner, B. Buesser, A.-M. Creţu, E. Debenedetti, D. Dobos, D. Fabian, M. Fischer, D. Froelicher, K. Grosse, D. Naeff, E. Ozoani, A. Paverd, F. Tramèr, and V. Volhejn, "Design patterns for securing LLM agents against prompt injections," arXiv preprint arXiv:2506.08837, 2025. [Online]. Available: https://arxiv.org/abs/2506.08837

[57] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Network and Distributed System Security Symposium (NDSS)*, 2025, accessed 2026-10-15. [Online]. Available: https://www.ndss-symposium.org/wp-content/uploads/2025-1131-paper.pdf

[58] T. Shi, J. He, Z. Wang, L. Wu, H. Li, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," arXiv:2504.11703, 2025, accessed 2026-10-15. [Online]. Available: https://arxiv.org/abs/2504.11703

[59] S. Willison, "The lethal trifecta for AI agents: Private data, untrusted content, and external communication," Simon Willison's Weblog, Jun. 2025, accessed 2026-10-15. [Online]. Available: https://simonwillison.net/2025/Jun/16/the-lethal-trifecta/

[60] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," arXiv preprint arXiv:2503.23278, 2025. [Online]. Available: https://arxiv.org/abs/2503.23278

[61] B. Radosevich and J. Halloran, "MCP safety audit: LLMs with the model context protocol allow major security exploits," arXiv preprint arXiv:2504.03767, 2025. [Online]. Available: https://arxiv.org/abs/2504.03767

[62] M. M. Hasan, H. Li, E. Fallahzadeh, G. K. Rajbahadur, B. Adams, and A. E. Hassan, "Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers," arXiv preprint arXiv:2506.13538, 2025. [Online]. Available: https://arxiv.org/abs/2506.13538

[63] Z. Wang, Y. Gao, Y. Wang, S. Liu, H. Sun, H. Cheng, G. Shi, H. Du, and X. Li, "MCPTox: A benchmark for tool poisoning on real-world MCP servers," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 42, 2026, pp. 35 811–35 819. [Online]. Available: https://ojs.aaai.org/index.php/AAAI/article/view/40895

[64] Trail of Bits, "Jumping the line: How MCP servers can attack you before you ever use them," Trail of Bits blog, Apr. 2025, accessed 2026-10-15. [Online]. Available: https://blog.trailofbits.com/2025/04/21/jumping-the-line-how-mcp-servers-can-attack-you-before-you-ever-use-them/

[65] J. Rehberger, "Wrap up: The month of AI bugs," Embrace The Red blog, Aug. 2025, accessed 2026-10-15. [Online]. Available: https://embracethered.com/blog/posts/2025/wrapping-up-month-of-ai-bugs/

[66] W. Vandevanter, "Prompt injection to RCE in AI agents," Trail of Bits blog, Oct. 2025, accessed 2026-10-15. [Online]. Available: https://blog.trailofbits.com/2025/10/22/prompt-injection-to-rce-in-ai-agents/

[67] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium (USENIX Security 22)*, Boston, MA, 2022, accessed 2026-10-15. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[68] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium (USENIX Security 23)*, Anaheim, CA, 2023, pp. 6983–7000, accessed 2026-10-15. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[69] M. Ohm, H. Plate, A. Sykosch, and M. Meier, "Backstabber's knife collection: A review of open source software supply chain attacks," arXiv:2005.09535, https://arxiv.org/abs/2005.09535, 2020, accessed 2026-10-15.

[70] OWASP GenAI Security Project, "Agentic AI — threats and mitigations," OWASP Foundation, Feb. 2025, accessed 2026-10-15. [Online]. Available: https://genai.owasp.org/resource/agentic-ai-threats-and-mitigations/

[71] OWASP Foundation, "OWASP MCP top 10," OWASP project page, accessed 2026-10-15. [Online]. Available: https://owasp.org/www-project-mcp-top-10/